Systems Administration (ASIST)

Practical Theoretical Class 11

Clustering: failover and load balancing.
Linux – HAProxy
Windows Server - NLB (*Network Load Balancing*)

Fault tolerance

Fault tolerance is the property of a system that allows it to continue to function normally even if one or more of its components fail.

Fault tolerance typically implies redundancy, ie having several identical components that perform the same function. It also implies mechanisms that detect failing components and ensure that the system stops using them without impacting performance.

Few systems are fully fault tolerant because they would have to eliminate any single point of failure (**SPoF** – *Single Point of Failure*).

For example, normally a RAID1 disk system is considered fault tolerant, but this is true only from the point of view of the disk component, not from the point of view for example the disk controller.

Failover

One approach to providing fault tolerance for a component of a system is ***failover***.

In the failover approach there are several similar components (redundancy), however only one of them is active (to be used by the system), the rest are in standby, but ready to be activated.

If the operating component fails, one of the standby components is automatically activated by a control system to replace it.

The process of replacing the failed component must be automatic and rapid to avoid any disruption to overall system operation. The control system that monitors the status of the active component and triggers its replacement is usually an undesirable single point of failure.

If the component is stateful then it is also necessary to implement synchronization mechanisms that ensure that standby components at all times have the same state as the failing component (example: **RAID1**).

Load Balancing

Having standby components is always a waste of resources. If it is possible for the system to simultaneously use the various redundant components to improve their performance while ensuring fault tolerance this is the ideal option. It is commonly known as **load balancing.**

With *load balancing* all redundant components are in permanent operation using an appropriate control mechanism. For example, the system can alternately use each component if a component fails (*fault tolerance*).

The way the control mechanism defines which component will be used at any given moment can be more or less sophisticated, from simple ***round robin*** to performance monitoring of each component and use of the component that currently offers the best guarantees.

Note that component status monitoring is generally required to detect faults. Again, the control system is a single point of failure.

Networks - *Fault Tolerance / Load Balancing*

We already know about mechanisms that guarantee fault tolerance and load balancing in networks.

Level 2 (ex.: Ethernet) – If redundant paths or links exist between Level 2 intermediate nodes (switches), *Spanning Tree Protocol* (**STP**) implements *failover*, *Link Aggregation Control Protocol* (**LACP**) or other *Link Aggregation* equivalents implement *load balancing* with *fault tolerance*.

The *Link Aggregation* technique can also be used to bind end nodes as servers. For Windows servers through a ***NIC Team***, Linux servers via a ***bonded NIC***.

Level 3 (ex.: IPv4/IPv6) – If redundant paths exist between level 3 intermediate nodes (routers), dynamic routing supported by routing protocols allows to implement load balancing with fault tolerance. All routing protocols guarantee *fault tolerance*, *load balancing* efficiency depends essentially on the metric used.

Servers and network services

In implementing redundant network services, the strategy that should be adopted is the use of redundant servers. This stems from the need to avoid single points of failure as much as possible.

If redundant network services are implemented using the same server, in case of server failure all alternative service provision will fail. It would also be necessary to deal with the impossibility of having multiple redundant services on the same server using the same port number.

In the context of server virtualization, we must also choose not only to place redundant services on different servers, but also preferably on virtual servers residing on different and independent virtualization platforms.

Again the motivation is to avoid single points of failure that affect all alternative services, in which case if the virtualization platform fails, all servers that are there and consequently all alternative services fail.

Server *clusters*

In short, when implementing redundant network services, we should avoid having the possibility of all being affected by the same failure. This is accomplished by deploying them on separate machines and preventing single points of failure that could affect all machines.

A *server cluster* is a set of servers that work together for a common purpose. Typical goals of a server cluster are *load balancing* and *fault tolerance*, ie providing high-availability cluster (HA-cluster) services.

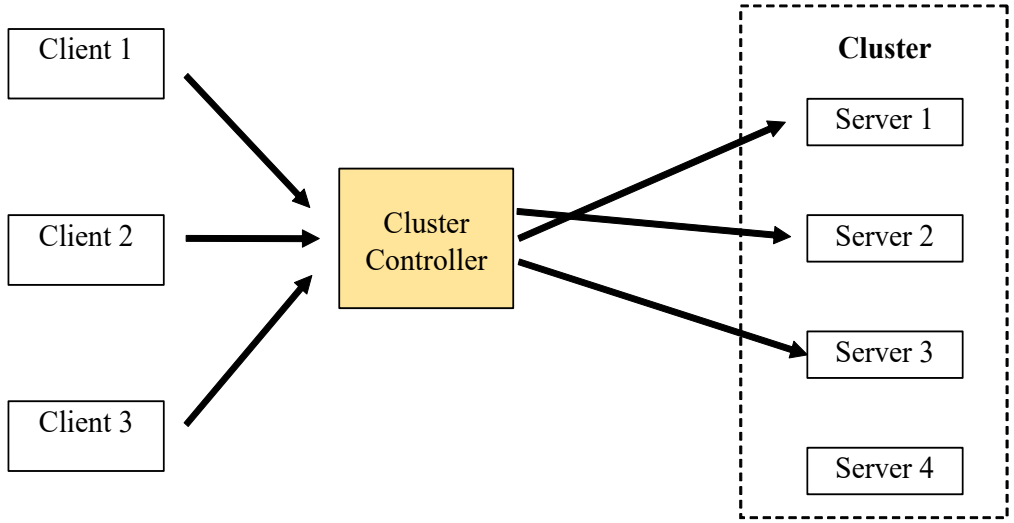
Another feature that a server cluster must have is that it should be used as a *single system*, that is, requests are directed to the cluster and not to individual servers in the cluster.

Often this feature is implemented through a cluster controller that is a single point of access to it, hence also a single point of failure.

Single *server cluster* access IP

Although it is a single point of failure, one way to implement a single point of access to a server cluster is through a controller with a unique IP address.

Clients access the cluster by sending requests to the controller's unique IP address, the controller redirects requests to the various cluster servers according to *load balancing* and *fault tolerance* criteria.

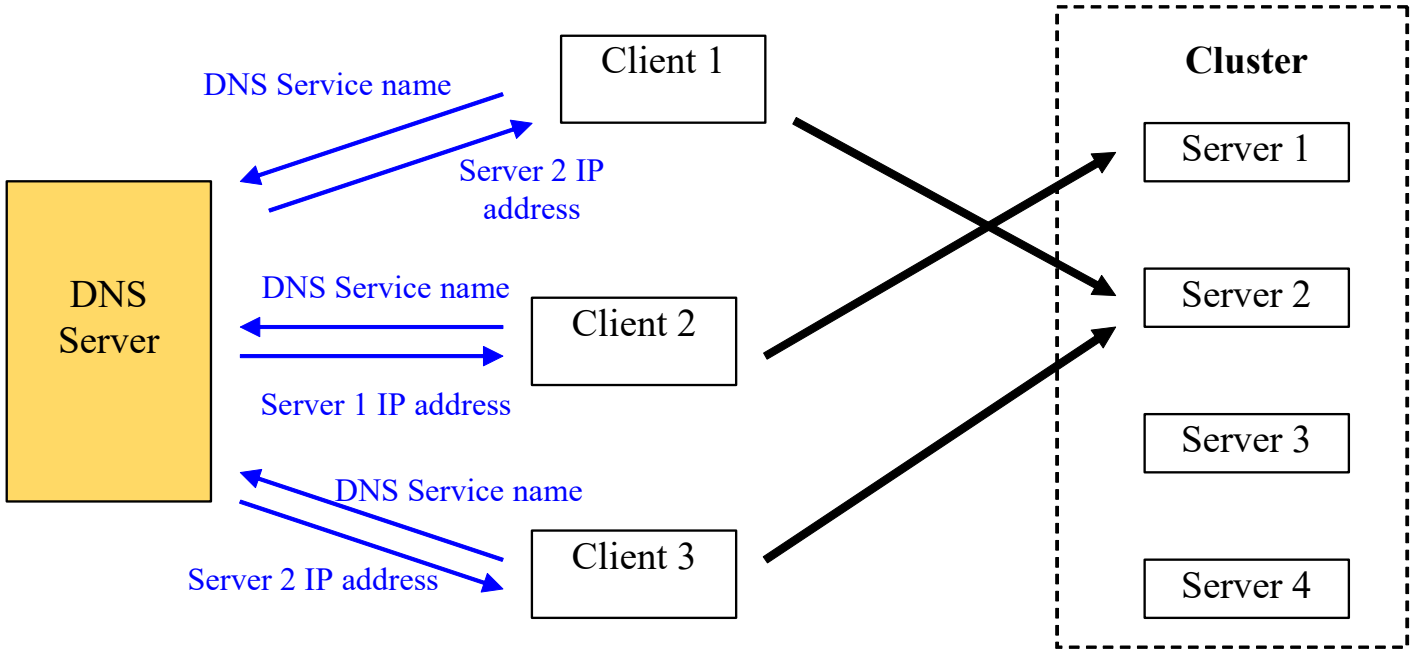


The controller can be deployed through a router running *Destination Network Address Translation* (**DNAT**) or a proxy server by simply redirecting the requests it receives.

For many application protocols it will be desirable that once a server is assigned to a client this assignment will be maintained (eg HTTP cookies). Otherwise a tighter synchronization between the various servers will be required.

Server cluster access unique name

An alternative to a single IP is the use of a unique DNS name, the advantage is that this system relies on fault-tolerant DNS operation..



The cluster controller is the DNS server, it is the one that distributes clients across the various cluster servers according to *load balancing* and *fault tolerance* criteria. DNS records provided by the DNS server to clients must necessarily have a very short lifespan (**TTL**) as long as they do not expire they will continue to be used without re-querying the server.

Failover and *load balancing* by clients

One way to avoid the single point of cluster access failure is to violate one of the characteristics a cluster must have and allow clients to access individual servers in the cluster. In this case control passes to the client.

The client has a list of alternate servers and implements *failover* itself, ie tries to use one of them.

In addition to *failover*, the client can also implement a basic *load balancing* form by rotating the list (*round-robin*).

When implementing failover by the client, it is essential to properly define the maximum timeout for server access attempts.

This method is widely used in various circumstances such as DNS clients, SMTP clients, LDAP clients (**nss_ldap**), etc..

DNS servers can be used to centralize this process through **MX** (**SMTP**), **SRV**, and **A**/**AAAA** records.

DNS – Round-robin

In DNS we can define the same records of type NS, MX, SRV, A and AAAA, ie they associate the same DNS name with different data. Example:

```
ssh.dei.isep.ipp.pt.   60   A     193.136.62.80
ssh.dei.isep.ipp.pt.   60   A     193.136.62.81
ssh.dei.isep.ipp.pt.   60   A     193.136.62.82
ssh.dei.isep.ipp.pt.   60   A     193.136.62.83
ssh.dei.isep.ipp.pt.   60   A     193.136.62.84
ssh.dei.isep.ipp.pt.   60   A     193.136.62.85
```

In this example there are 6 A records all with the same name, when a client requests DNS name resolution **ssh.dei.isep.ipp.pt** will get a list of 6 IP addresses.

However, in view of load balancing, the DNS server does not provide them in the same order to all clients, but runs the list between requests (*round-robin*).

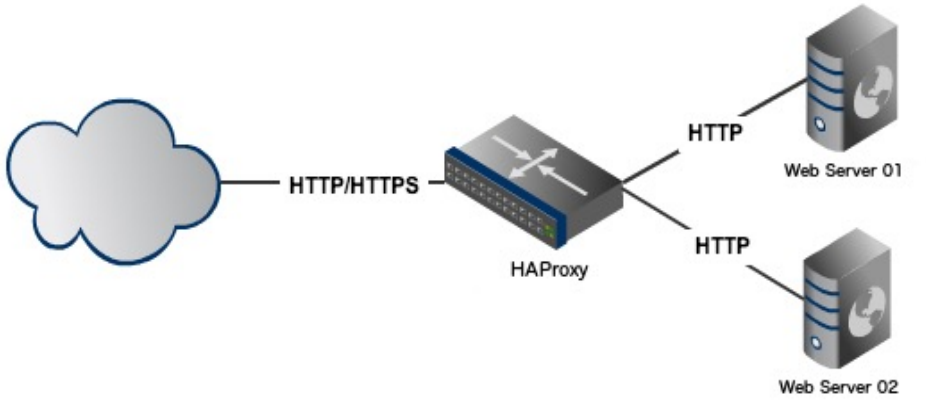
Assuming that each client first tries to use the first address from the list provided, this leads to clients being distributed across multiple servers.

This feature should be implemented by all DNS servers whenever records with repeated names exist.

Linux – HAProxy (The Reliable, High Performance TCP/HTTP Load Balancer)

HAProxy (*http://www.haproxy.org/*) is a Linux cluster controller capable of redirecting TCP connections to a server cluster including security features and functionality specific to the HTTP protocol.

Among other features, HAProxy allows for example to receive SSL/TLS protected connections as HTTPS and to transform them into non-secure connections within the cluster.



This is useful because as clients access the cluster through the DNS name of the host HAProxy is in, this avoids having to keep the same public key certificates on all cluster servers.

HAProxy enables to implement a variety of security functions ranging from limiting the number of connections per second to validating HTTP header lines before being relayed to cluster servers.

Frontends and backends

In HAProxy a **frontend** represents an access point for clients, a **backend** is a group of cluster servers.

The configuration file consists of sections containing configuration parameters, namely a **global** section and several **defaults**, **frontend**, **backend** and **listen** sections.

Declaration section	
global	Sets global parameters related to the overall functioning of HAProxy.
defaults [NAME]	Defines parameter sets that will apply to all following sections unless changed in the section or through another defaults declaration.
frontend NAME	Defines a *frontend*, namely through the **bind** parameter and the **use_backend** parameter.
backend NAME	Defines a *backend*, namely through **server** parameters.
listen NAME	Most expeditious declaration form of a proxy including in the same *frontend* and *backend* declaration.

Stickiness

When HAProxy is contacted by a client, the corresponding **frontend** will be associated with a **backend**, among the various servers defined in the **backend** will be assigned one to that client.

Due to the functioning of the application protocols, HAProxy must maintain this ***server<->server*** association on subsequent requests, this is called ***stickiness***.

The *stickiness* mechanism is managed automatically by *stick-tables*, but there are several parameters that allow its control.

Parameters ***mode tcp*** and ***mode http***

For each frontend/backend, two modes of operation are supported: **tcp** and **http**. tcp mode relays information directly between client and server without parsing it. In http mode, HTTP messages are parsed and validated before being transmitted to the server.

Parameter balance ALGORITHM

The **balance** parameter can be used in the **backend** and **listen** sections, it is used to define the load balancing mode between the active servers that are defined in the **backend.**

There are about nine algorithms available, most of which are parameterizable. If none is set, the **roundrobin** value is used. The **roundrobin** algorithm is dynamically weighted, ie the probability of a server being selected depends on its previous efficiency.

Another possible algorithm is **leastconn**, in which case the elected one is the server with the least number of currently active connections. Between multiple servers with the same number of bindings, round robin is applied.

Note that the algorithm is only used when it is an unknown client, missing from the *stick-table* or when there is a need to assign it a new server because the previous one is no longer available.

HAProxy – configuration example

The figure represents a *listen* section of the configuration file.:

```
listen http_proxy
     bind 10.2.0.1:80,10.2.0.1:8080
     stats enable
     mode http
     option httpclose
     option forwardfor
     balance roundrobin
     server S1 192.168.5.10:80 check
     server S2 192.168.5.11:80 check backup
     server S2 192.168.5.12:80 check
```

Provides statistical information via HTTP on URL /haproxy?stats

Inserts in HTTP headers Connection: close

Insert in HTTP headers X-Forwarded-For: IP-CLIENTE

The **check** option in the server statement forces periodic checking of its status over TCP connections..

A server declared as **backup** is only used if none of the others are available..

Windows Server - NLB and *clustering*

The Windows Server *Network Load Balancing* (**NLB**) feature allows the server to be integrated into a cluster called *NLB Cluster*.

Windows server NLB clusters take a different approach: all machines that are *cluster* members will have the same node address, which will be the cluster address.

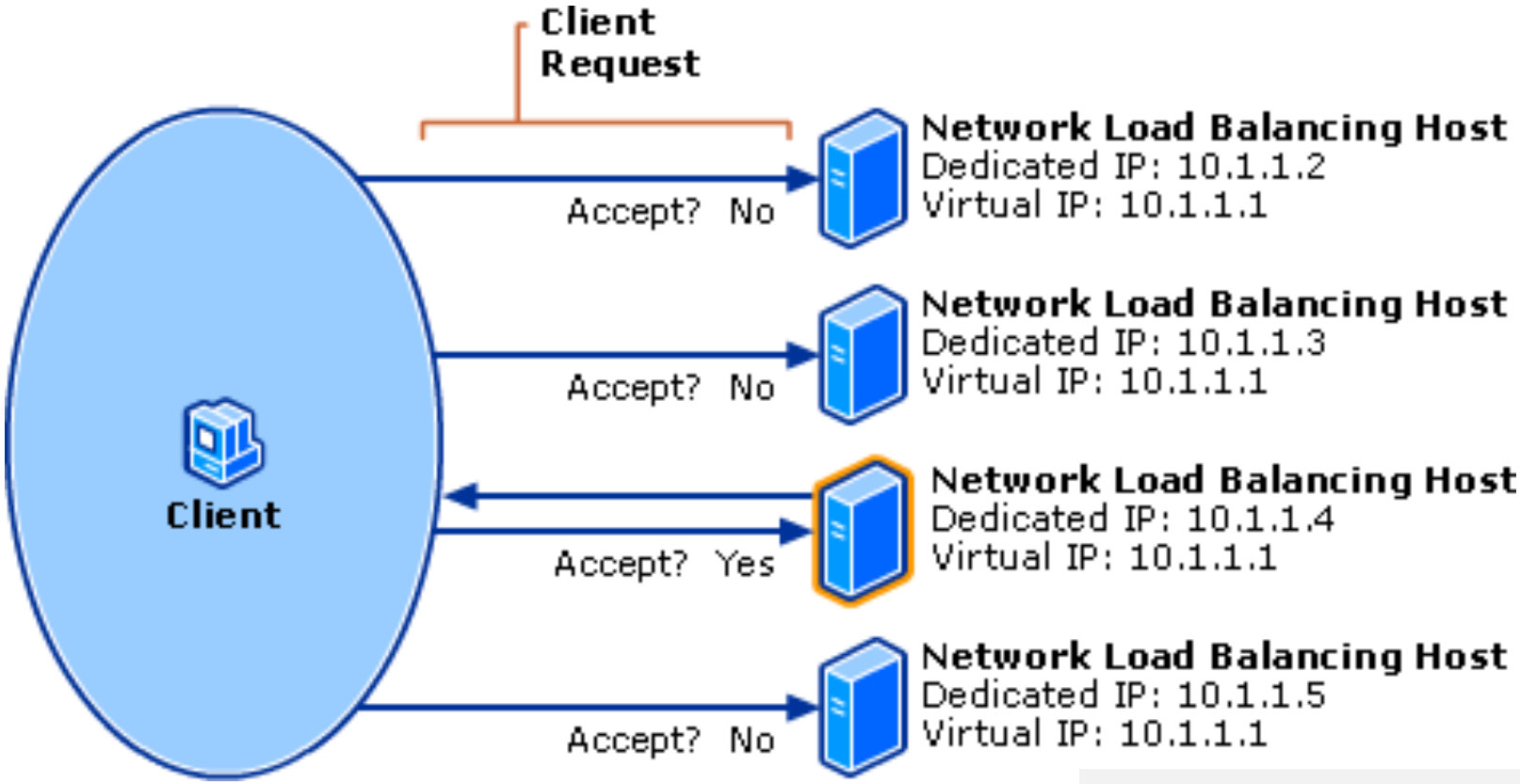
When a client sends a request to the cluster address, all machines that are members of the cluster will receive a copy of the request. Only one member can answer, that is established between the various cluster members.

If a member is no longer available, requests continue to be answered by other members.

The great advantage of NLB clusters is that there is no single point of failure (SPOF), there is no centralized cluster control, it is managed distributed across all members. As long as at least one member is working, the cluster is available.

NLB *Clusters* – Network operation

In an NLB Cluster all members have the same node address, but only one member responds to each request.



(Source: technet.microsoft.com)

Cluster members can keep their unique IP address (*Dedicated IP*), but they will all share a common (*Primary / Virtual IP*) address that will be the NLB Cluster address.

NLB *Clusters* – Level 2 Unicast

Two ways are available to ensure that requests addressed to the NLB cluster address reach all members:

- Level 2 Unicast (MAC)
- Level 2 multicast (MAC)

Microsoft's recommended approach is level 2 unicast. In this configuration, the level 2 (MAC/Ethernet) addresses of all cluster members' network interfaces are replaced with a single address equal to all members of the cluster (***inbound destination address***).

Frames issued by cluster members never use this address as their source as this would cause network switches to include it in their MAC table and subsequent requests would no longer be delivered to all cluster members.

When a cluster member issues a response, it uses as its source MAC address the ***outbound source address*** that is unique to each cluster member.

NLB *Clusters* – Level 2 Multicast

This approach is similar to the previous one, the only difference is that multicast MAC addresses are used.

On the network interfaces of all cluster members a level 2 (MAC/Ethernet) multicast address is added, equal to all members (*inbound destination address*). Since it is a multicast address, the original interface address can remain. Being a multicast MAC address there are no problems with the operation of network switches because they are always required to relay these frames on all their ports.

The attached table shows the MAC addresses used. While the *inbound* address is the same for all cluster members, *source outbound* addresses contain the value P that is unique to each member.

IP Mode	MAC Address	Explanation
Unicast inbound	02-BF-W-X-Y-Z	W-X-Y-Z = IP address Onboard MAC disabled
Multicast inbound	03-BF-W-X-Y-Z	W-X-Y-Z = IP address Onboard MAC enabled
Source outbound	02-P-W-X-Y-Z	W-X-Y-Z = IP address P = Host priority

(Source: technet.microsoft.com)

NLB *Clusters* – modes

Implementing an NLB cluster can never combine different forms of operation, all cluster members operate the same, unicast or multicast.

Whatever the option, the fact that all cluster members use the same IP address requires that they are all on the same level 2 network (broadcast domain).

In **unicast mode** the original MAC address of the network interface is disabled and replaced by an address of the form 02: bf:…, common to all members, as a result of disabling the original MAC address the member is no longer accessible individually and is only accessible as a cluster. To be able to communicate individually with a member of a unicast cluster you must provide the member with an additional network interface, otherwise to individually access a member you must first remove them from the cluster.

In **multicast mode** the previous problem does not occur because a multicast MAC address can be added to the interface without disabling the original MAC address. Thus the same network interface can be used for cluster virtual address and member dedicated address.

NLB *Clusters* – multicast with IGMP

Due to the way they operate with frames, ensuring that they reach all cluster members, NLB clusters prevent segmentation by switches.

An NLB cluster floods the entire framed broadcast domain, so ideally limit the broadcast domain as much as possible, ie the cluster network should preferably be a dedicated network.

In multicast mode the *Internet Group Membership Protocol* (**IGMP**) can be used. Using the ICMP option the *inbound destination address multicast* is defined differently and members issue IGMP messages to register for multicast groups.

Using IGMP is only of interest if in the broadcast domain there are network switches that support it. Through IGMP messages these switches prevent the propagation of multicast frames to unnecessary locations.

NLB *Clusters* - ARP

When a node located on the cluster network (typically the router on that network) tries to access the primary or virtual address (IPv4) cluster address, it will try to obtain the corresponding MAC address through *Address Resolution Protocol* (**ARP**). How the NLB cluster uses MAC addresses can then cause problems.

When cluster members respond, the ARP response contains the *destination inbound unicast* or *multicast* MAC address (same for all members), however the ARP response is carried in a frame that has the *source outbound* MAC address (unique for each member).

This may be interpreted by some nodes as inconsistent and may cause the node to ignore the ARP response for security reasons.

One possible solution to this situation is to insert a static entry into the ARP table on these nodes to map the destination inbound unicast or multicast MAC address to the cluster's primary or virtual address.

NLB *Clusters* – Priority

In the current implementation, an NLB cluster can contain up to 32 members, each member must use a different priority number from 1 to 32.

The priority number is set by the administrator when adding a member to the cluster, the value 1 means higher priority. Depending on port rules, the highest priority member can receive all requests from clients.

The priority number also identifies the member in the cluster, for example being placed in the *source outbound* MAC addresses of frames issued by that member.

The NLB cluster aims to receive TCP and UDP traffic destined for network services and to distribute this traffic among its members. The way this distribution is performed is defined in the *port rules*.

Port rules only apply to UDP and TCP, for example ICMP traffic is always sent to all cluster members (only the highest priority responds to requests).

NLB *Clusters – Port rules*

A cluster can have multiple addresses (*primary or virtual address*), for each of them different port rules can be defined, however all members of a cluster must have the same port rules, when you add a new member it acquires the cluster ports rules.

To change the port rules in a member it must be removed from the cluster, the rules changed and added again, these operations can be performed without disturbing the cluster operation from the users point of view.

For each cluster address we can define multiple port rules, each specifying a range of UDP and / or TCP port numbers. Ranges in different rules applied to the same address cannot overlap.

The purpose of a port rules is to define how to handle the corresponding traffic arriving at the cluster, there are three alternatives (*filtering modes*):

- ***Multiple Host*** – traffic is distributed by members
- ***Single Host*** –*failover* mode
- ***Disable*** – traffic is not allowed

NLB *Clusters – filtering mode single host*

The default mode is *single host*, ie it applies to any traffic that does not match any *port rule*. This includes traffic that cannot be defined in a *port rule*, such as ICMP.

Filtering mode ***single host*** corresponds to ***failover*** behavior, all traffic is directed to the highest priority available member, if it becomes unavailable the next higher priority available member will be used. Once added to the cluster, for each member you can set a specific priority value (1 through 32) for each *port rule*.

Filtering mode ***multiple host*** allows traffic to be distributed across all available members.

In this mode it is also possible to define for each member the percentage of requests addressed to them, by default the requests are distributed evenly.

Setting the percentage of requests assigned to each member can only be done on member properties once added to the cluster and not on cluster properties.

NLB *Clusters – multiple host and affinity*

In *filtering mode* ***multiple host*** it is essential to define the persistence of associations between clients and cluster members. *Affinity* allows you to ensure that when a customer's first order is assigned to a member, subsequent orders from that same customer will also be assigned to the same member. Without this guarantee many application protocols such as those operating over TCP cannot work.

The available *infinity* modes are:

none – all orders are distributed by members, can only be used for very simple unbound services. For example UDP services with one request and one response.

single – all orders coming from the same source (client) address are sent to the same member.

network – all requests coming from the same class C network (24-bit prefix) are sent to the same member.

In ***single*** and ***network*** modes a timeout can be specified in minutes. If set, when the client does not communicate during that time period, persistence is eliminated and the next communication will be assigned a new member to the client.

Cluster Member Synchronization

Because members of a cluster should appear to be a single server to clients, the services provided by each cluster must have synchronization mechanisms that ensure all members have the same state.

Affinity, or *stickiness*, ensures that during a client's work session he will always work with the same member, but that may not be enough.

For example, when a user uploads content to a web site that is clustered, they expect that content to be available to all users in the site/cluster.

Cluster member synchronization should be implemented on a case by case basis for each type of service provided, for example network file systems (NAS) can be used for this purpose, database servers usually have their own replication mechanisms.

Public Key Certificate Synchronization

Desirably, from a security standpoint, many application protocols currently use SSL/TLS. In this context the server provides the client with a public key certificate to be validated by the client for server authentication purposes.

The public key certificate contains the server's DNS name in a cluster for the certificate to be considered valid by the client, this DNS name must match the cluster's IP address (address used by clients in cluster access).

Consequently, all cluster members will need to use this same certificate.

In the case of HAProxy the installation and maintenance of the same certificate on all members can be avoided. When HAProxy operates in HTTP mode, it can itself provide the certificate to the client, in which case the link between the client and HAProxy uses HTTPS while the corresponding HAProxy link to the member can use HTTP.